UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Blade Arena Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Blade Arena

Blade Arena is a standalone editable Unity game template. Drag to steer your knight or use WASD or arrow keys on a keyboard. The blade trails behind your turns and can hit enemies while you circle. Tap Slash or press Space for a cooldown-based Spin Slash. Clear every enemy wave before losing all health. Fences, arena edges and traps affect positioning. The included AI opponents run locally.

Includes 25 curated levels, level generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Blade Arena > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/BladeArena/Scenes/BladeArena_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/BladeArena`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/bladearena contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/BladeArenaGame.cs`
- `Scripts/BladeArenaGame.UI.cs`
- `Scripts/BladeArenaGame.View.cs`
- `Scripts/BladeData.cs`
- `Scripts/BladeGen.cs`
- `Scripts/BladeLevels.cs`
- `Scripts/BladeMeshes.cs`
- `Scripts/BladeSfx.cs`
- `Scripts/BladeSim.cs`

Main flow: BladeArenaGame.cs. UI and popup consumers: BladeArenaGame.UI.cs. Model/generator/loading: BladeSim.cs, BladeGen.cs, BladeLevels.cs. Authoring: Editor/BladeArenaLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag to steer your knight or use WASD or arrow keys on a keyboard. The blade trails behind your turns and can hit enemies while you circle. Tap Slash or press Space for a cooldown-based Spin Slash. Clear every enemy wave before losing all health. Fences, arena edges and traps affect positioning. The included AI opponents run locally.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. BoostName, BoostUnlock, BoostStart, BoostDesc and duration constants are in BladeArenaGame.cs; the HUD consumes them in BladeArenaGame.UI.cs.

Helper	Unlock	Free charges	Action and effect
Potion	3	3	Restores up to 50 health. Use while injured.
Whirlwind	6	2	Spins the blade for 2.6 seconds with stronger hits.
Aegis	10	2	Grants invulnerability for four seconds and pushes foes away.
Time Warp	15	2	Slows foes and traps for five seconds.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. Potion needs missing health to demonstrate healing; show its before and after health bar. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Blade Arena > Level Editor**. Exact implementation: `Editor/BladeArenaLevelEditor.cs`.

Duplicate an early arena in the level editor. Change the visible name and one wave or obstacle. Check the arena shape, boundary and spawn locations before saving. Wave and AI profile controls are listed below. Validate the edited arena and try its actual combat flow.

1. Back up Resources/bladearena/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`BladeArenaLevelEditor.cs:47`	`[MenuItem("Szekipapa77/Blade Arena/Level Editor")]`
`BladeArenaLevelEditor.cs:50`	`[MenuItem("Szekipapa77/Blade Arena/Validate All Levels")]`
`BladeArenaLevelEditor.cs:118`	`if (GUILayout.Button(_dirty ? "Save*" : "Save", EditorStyles.toolbarButton, GUILayout.Width(60))) Save();`
`BladeArenaLevelEditor.cs:119`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`BladeArenaLevelEditor.cs:122`	`if (GUILayout.Button("Validate level", EditorStyles.toolbarButton, GUILayout.Width(95)) && Cur != null) _results[_sel] = BladeGen.Validate(Cur, _file.profiles, _valSeeds);`
`BladeArenaLevelEditor.cs:123`	`if (GUILayout.Button("Validate all", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAllInWindow();`

BladeArenaLevelEditor.cs:125	`if (GUILayout.Button(new GUIContent("Play this level", "Quick test only: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(100)) && Cur != null) Play(_sel + 1);`
BladeArenaLevelEditor.cs:155	`if (GUILayout.Button("Add")) { var l = NewLevel(); Insert(_sel + 1, l); }`
BladeArenaLevelEditor.cs:156	`if (GUILayout.Button("Dup") && Cur != null) { var c = Cur.Clone(); c.name += " copy"; Insert(_sel + 1, c); }`
BladeArenaLevelEditor.cs:157	`if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))`
BladeArenaLevelEditor.cs:163	`if (GUILayout.Button("Up") && _sel > 0) Swap(_sel, _sel - 1);`
BladeArenaLevelEditor.cs:164	`if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Swap(_sel, _sel + 1);`
BladeArenaLevelEditor.cs:193	`l.name = EditorGUILayout.TextField("Name", l.name);`
BladeArenaLevelEditor.cs:194	`l.theme = EditorGUILayout.Popup("Theme", l.theme, Themes.Names);`
BladeArenaLevelEditor.cs:197	`l.fenced = EditorGUILayout.ToggleLeft("Fenced (no ring-out)", l.fenced, GUILayout.Width(170));`
BladeArenaLevelEditor.cs:198	`l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(90));`
BladeArenaLevelEditor.cs:200	`ii = EditorGUILayout.Popup("Introduces", ii, Intros.Select(s => s == "" ? "(none)" : s).ToArray());`
BladeArenaLevelEditor.cs:202	`l.difficulty = EditorGUILayout.Slider("Design difficulty", l.difficulty, 0, 1);`
BladeArenaLevelEditor.cs:204	`l.hint = EditorGUILayout.TextField("Start tip", l.hint);`

BladeArenaLevelEditor.cs:208	`int nc = EditorGUILayout.IntSlider("Columns", l.cols, 4, 13);`
BladeArenaLevelEditor.cs:209	`int nr = EditorGUILayout.IntSlider("Rows", l.rows, 4, 13);`
BladeArenaLevelEditor.cs:211	`l.shape = EditorGUILayout.TextField("Shape tag", l.shape, GUILayout.Width(260));`
BladeArenaLevelEditor.cs:221	`_brushWave = EditorGUILayout.IntSlider("Wave", _brushWave + 1, 1, Mathf.Max(1, l.waves.Length + 1)) - 1;`
BladeArenaLevelEditor.cs:222	`_brushProfile = EditorGUILayout.Popup("Profile", _brushProfile, _file.profiles.Select((p, i) => $"{i} {p.name}").ToArray());`
BladeArenaLevelEditor.cs:224	`else if (_brush == Brush.Trap) _trapSpeed = EditorGUILayout.Slider("Trap speed (deg/s, sign = direction)", _trapSpeed, -300, 300);`
BladeArenaLevelEditor.cs:375	`EditorGUILayout.LabelField("Summary", EditorStyles.boldLabel);`
BladeArenaLevelEditor.cs:376	`EditorGUILayout.LabelField($"Enemies: {l.EnemyCount()} in {l.waves.Length} wave(s)");`
BladeArenaLevelEditor.cs:377	`EditorGUILayout.LabelField($"Traps: {l.traps.Length} Barrels: {l.barrels.Length}");`
BladeArenaLevelEditor.cs:378	`EditorGUILayout.LabelField("Mechanics: " + string.Join(", ", l.Mechanics(_file.profiles)));`
BladeArenaLevelEditor.cs:379	`EditorGUILayout.LabelField($"Threat estimate: {BladeGen.Threat(l, _file.profiles):0.0}");`
BladeArenaLevelEditor.cs:384	`EditorGUILayout.LabelField($"Bot result ({v.Runs} runs)\n{(v.Solvable ? "Winnable" : "TOO HARD / BROKEN")}\nwin rate {v.WinRate * 100:0}%\navg time {v.AvgTime:0.0}s\nhealth left {v.AvgHp * 100:0}%\ndifficulty score {v.Difficulty:0.00}", st, GUILayout.Height(110));`
BladeArenaLevelEditor.cs:386	`else EditorGUILayout.HelpBox("Press 'Validate level' to run the bot (it plays the fight several times with different seeds).", MessageType.Info);`

BladeArenaLevelEditor.cs:388	`if (issues.Count > 0) EditorGUILayout.HelpBox(string.Join("\n", issues), MessageType.Warning);`
BladeArenaLevelEditor.cs:408	`_showWaves = EditorGUILayout.Foldout(_showWaves, "Enemy roster per wave", true);`
BladeArenaLevelEditor.cs:415	`EditorGUILayout.LabelField($"Wave {w + 1}", EditorStyles.boldLabel, GUILayout.Width(80));`
BladeArenaLevelEditor.cs:416	`if (GUILayout.Button("Remove wave", GUILayout.Width(100))) { var list = l.waves.ToList(); list.RemoveAt(w); l.waves = list.ToArray(); Changed(); GUIUtility.ExitGUI(); }`
BladeArenaLevelEditor.cs:424	`sp[k].x = EditorGUILayout.IntField("x", sp[k].x, GUILayout.Width(120));`
BladeArenaLevelEditor.cs:425	`sp[k].y = EditorGUILayout.IntField("y", sp[k].y, GUILayout.Width(120));`
BladeArenaLevelEditor.cs:427	`if (GUILayout.Button("x", GUILayout.Width(22))) { var list = sp.ToList(); list.RemoveAt(k); l.waves[w].spawns = list.ToArray(); Changed(); GUIUtility.ExitGUI(); }`
BladeArenaLevelEditor.cs:432	`if (GUILayout.Button("Add wave", GUILayout.Width(120))) { l.waves = l.waves.Append(new WaveData()).ToArray(); _brushWave = l.waves.Length - 1; _brush = Brush.Enemy; Changed(); }`
BladeArenaLevelEditor.cs:438	`EditorGUILayout.HelpBox("AI profiles drive every enemy. Levels reference them by index, so add new ones at the end.", MessageType.Info);`
BladeArenaLevelEditor.cs:452	`p.armor = EditorGUILayout.IntSlider("Armour", p.armor, 0, 2);`
BladeArenaLevelEditor.cs:453	`EditorGUILayout.LabelField($"threat {p.Threat:0.00}", GUILayout.Width(80));`
BladeArenaLevelEditor.cs:456	`p.hp = EditorGUILayout.FloatField("HP", p.hp); p.dmg = EditorGUILayout.FloatField("Damage", p.dmg); p.speed = EditorGUILayout.FloatField("Speed", p.speed); p.scale = EditorGUILayout.FloatField("Size", p.scale);`

BladeArenaLevelEditor.cs:459	p.windup = EditorGUILayout.Slider("Wind-up", p.windup, 0.15f, 1.5f); p.cooldown = EditorGUILayout.Slider("Cooldown", p.cooldown, 0.3f, 3f);
BladeArenaLevelEditor.cs:462	p.aggression = EditorGUILayout.Slider("Aggression", p.aggression, 0, 1); p.smart = EditorGUILayout.Slider("Hazard smarts", p.smart, 0, 1);
BladeArenaLevelEditor.cs:466	if (GUILayout.Button("Duplicate", GUILayout.Width(90))) { _file.profiles = _file.profiles.Append(p.Clone()).ToArray(); _dirty = true; GUIUtility.ExitGUI(); }
BladeArenaLevelEditor.cs:469	if (GUILayout.Button(new GUIContent("Delete", used ? "Used by a level" : "Only the last unused profile can be deleted"), GUILayout.Width(70))) { _file.profiles = _file.profiles.Take(_file.profiles.Length - 1).ToArray(); _dirty = true; GUIUtility.ExitGUI(); }
BladeArenaLevelEditor.cs:474	if (GUILayout.Button("Reset to default roster", GUILayout.Width(200)) && EditorUtility.DisplayDialog("Reset", "Replace all AI profiles with the defaults?", "Reset", "Cancel")) { _file.profiles = BladeGen.DefaultProfiles(); _dirty = true; }
BladeArenaLevelEditor.cs:481	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, bot-validated, same generator as the runtime)", true);
BladeArenaLevelEditor.cs:484	_genCount = EditorGUILayout.IntField("Count", _genCount); _genStart = EditorGUILayout.IntField("Start at level #", _genStart);
BladeArenaLevelEditor.cs:485	_genSeed = EditorGUILayout.IntField("Seed", _genSeed);
BladeArenaLevelEditor.cs:488	_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);
BladeArenaLevelEditor.cs:491	_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 2, 8);

`BladeArenaLevelEditor.cs:492`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`BladeArenaLevelEditor.cs:503`	`if (GUILayout.Button($"Generate {_genCount} level(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Set Count, Start at level #, Seed, Start difficulty, End difficulty and Breather every (2?8). Check Replace that range to remove that many existing entries from the chosen index before insertion; leave it unchecked to retain and shift later entries. Start at 26 to append to the initial bank. BladeGen varies shape/theme and uses the combat simulation for bot validation. Save after reviewing the generated levels.

Worked extension: back up the bank, generate five candidates after the original 25 at moderate difficulty, validate them, then Save. Reopen the JSON and confirm the count changed. An unchecked replace option inserts rather than silently destroying later data. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/BladeArena/Resources/bladearena/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "name": "Training Ring",
 "theme": 0,
 "shape": "square",
 "cols": 7,
 "rows": 7,
 "map": [
   "ooooooo",
   "ooooooo",
   "ooooooo",
   "ooooooo",
   "ooooooo",
   "ooooooo",
   "ooooooo"
 ],
 "fenced": true,
 "px": 3,
 "py": 5,
 "traps": [],
 "barrels": [],
 "waves": [
   {
     "spawns": [
       {
         "p": 0,
         "x": 3,
         "y": 1
       }
     ]
   }
 ],
 "intro": "duel",
 "hint": "",
 "breather": false,
 "difficulty": 0.05
}
```


Root keys: profiles, levels. Inspect BladeSim.cs, BladeGen.cs, BladeLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. AI profiles and wave placement are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/BladeArenaGame.UI.cs`. Find `Navy`, currently `#1B2340`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`BladeArenaGame.UI.cs:64 / void BuildHud()`	`var lp = HzUI.Panel("LevelPill", _hud, Navy, 44);`
`BladeArenaGame.UI.cs:76 / void BuildHud()`	`var set = HzIcons.IconButton(_hud, HzIcons.Settings, Navy, () => { if (!_ended) ShowSettingsPopup(); }, 108);`
`BladeArenaGame.UI.cs:592 / void ShowSettingsPopup()`	`var p = ArenaPopup("Settings", "", Navy, null,`

Source	Exact current code
`BladeArenaGame.cs:28 / class/static declarations`	`static readonly string[] BoostColor = { "#E8506A", "#1FB5A8", "#4C8DF0", "#9B6BFF" };`
`BladeArenaGame.cs:155 / IEnumerator Intro(string key)`	`ArenaPopup(it.title, it.msg, HzUI.Hex("#D9653B"), Art(it.art), ("Got it!", HzUI.Hex("#1FB5A8"), () => closed = true));`
`BladeArenaGame.cs:316 / void HandleEvents(List<Ev> evs)`	`FloatText(at + Vector3.up * 0.6f, "ARMOR!", HzUI.Hex("#9FB3CC"));`
`BladeArenaGame.cs:321 / void HandleEvents(List<Ev> evs)`	`if (e.A == 0 || e.B == 0) FloatText(at + Vector3.up * 0.4f, e.V == 0 && e.B == 0 ? "BLOCKED" : "PARRY!", HzUI.Hex("#FFE27A"));`
`BladeArenaGame.cs:376 / void HandleEvents(List<Ev> evs)`	`case EvKind.Heal: HzAudio.Play(Sfx.Heal, 0.85f); HealFx(); FloatText(PlayerWorld() + Vector3.up * 2f, "+" + Mathf.RoundToInt(e.V), HzUI.Hex("#5BE08A")); break;`

`BladeArenaGame.cs:451 / IEnumerator LoseFlow(bool fell)`	`if (!_reviveUsed) buttons.Add(("Revive (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "revive", ok => { if (ok) Revive(); else LoadLevel(Level, true); })));`
`BladeArenaGame.cs:452 / IEnumerator LoseFlow(bool fell)`	`buttons.Add(("Try Again", HzUI.Hex("#F2A93B"), () => LoadLevel(Level, true)));`
`BladeArenaGame.cs:453 / IEnumerator LoseFlow(bool fell)`	`if (_fails >= 2) buttons.Add(("Skip Level (ad)", HzUI.Hex("#9B6BFF"), () => HzPlatform.Rewarded(GameId, "skip_level", ok => { if (ok) { Level = Level + 1; LoadLevel(Level); } })));`
`BladeArenaGame.cs:454 / IEnumerator LoseFlow(bool fell)`	`ArenaPopup("Defeated!", msg + (_sim.EnemiesLeftTotal() > 0 ? $"\n{_sim.EnemiesLeftTotal()} knight{(_sim.EnemiesLeftTotal() == 1 ? "" : "s")} left standing." : ""), HzUI.Hex("#7A5CC9"), Art("lose_badge"), buttons.ToArray());`
`BladeArenaGame.cs:488 / void OnBooster(int b)`	`("Get 1 (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant().Replace(' ', '_'), ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),`
`BladeArenaGame.cs:489 / void OnBooster(int b)`	`("No thanks", HzUI.Hex("#B8ABD1"), null));`
`BladeArenaGame.UI.cs:13 / class/static declarations`	`static readonly Color Ink = HzUI.Hex("#2A2F4F");`
`BladeArenaGame.UI.cs:14 / class/static declarations`	`static readonly Color Navy = HzUI.Hex("#1B2340");`
`BladeArenaGame.UI.cs:15 / class/static declarations`	`static readonly Color Cream = HzUI.Hex("#FFF4DE");`
`BladeArenaGame.UI.cs:16 / class/static declarations`	`static readonly Color Teal = HzUI.Hex("#1FB5A8");`
`BladeArenaGame.UI.cs:67 / void BuildHud()`	`var lpEdge = HzUI.Panel("Edge", lp.transform, HzUI.Hex("#F2C14E"), 44); lpEdge.raycastTarget = false; lpEdge.sprite = HzUI.RoundRect(44, 5); lpEdge.rectTransform.Stretch(4);`

`BladeArenaGame.UI.cs:68 / void BuildHud()`	`_levelText = HzUI.Label("Lvl", lp.transform, "Level " + Level, 54, HzUI.Hex("#FFE7A8"));`
`BladeArenaGame.UI.cs:71 / void BuildHud()`	`_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#CFE9FF"), TextAnchor.MiddleCenter, false);`
`BladeArenaGame.UI.cs:85 / void BuildHud()`	`var heart = HzIcons.Icon(hp.transform, HzIcons.Heart, 66, HzUI.Hex("#FF5A6E")); heart.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(80, 80), new Vector2(56, 0));`
`BladeArenaGame.UI.cs:87 / void BuildHud()`	`var barBg = HzUI.Panel("BarBg", hp.transform, HzUI.Hex("#2E3558"), 22); barBg.raycastTarget = false;`
`BladeArenaGame.UI.cs:90 / void BuildHud()`	`_hpLag = HzUI.Panel("Lag", barBg.transform, HzUI.Hex("#FFD27A"), 22).rectTransform; Bar(_hpLag);`
`BladeArenaGame.UI.cs:91 / void BuildHud()`	`_hpFill = HzUI.Panel("Fill", barBg.transform, HzUI.Hex("#4BE08D"), 22).rectTransform; Bar(_hpFill);`
`BladeArenaGame.UI.cs:102 / void BuildHud()`	`var ei = HzIcons.Icon(_enemyChip, HzIcons.Groups, 58, HzUI.Hex("#FF9A6B")); ei.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(80, 80), new Vector2(54, 6));`
`BladeArenaGame.UI.cs:104 / void BuildHud()`	`_waveText = HzUI.Label("W", _enemyChip, "", 26, HzUI.Hex("#CFE9FF"), TextAnchor.MiddleCenter, false); _waveText.fontStyle = FontStyle.Bold;`
`BladeArenaGame.UI.cs:119 / void BuildHud()`	`_slashBtn = HzUI.Button("Slash", bar, null, HzUI.Hex("#F2C14E"), Color.white, null, 40, (int)(ss / 2));`
`BladeArenaGame.UI.cs:123 / void BuildHud()`	`var inner = HzUI.Panel("Inner", srt, HzUI.Hex("#D9653B"), 64); inner.raycastTarget = false; inner.rectTransform.Stretch(12);`
`BladeArenaGame.UI.cs:205 / static void SetBar(RectTransform r, float k)`	`var bd = HzUI.Panel("Badge", v.Root, HzUI.Hex("#FF4D6D"), 30); bd.raycastTarget = false;`
`BladeArenaGame.UI.cs:223 / void RefreshBoosters()`	`v.Inner.color = un ? HzUI.Hex(BoostColor[b]) : HzUI.Hex("#5A607E");`
`BladeArenaGame.UI.cs:228 / void RefreshBoosters()`	`v.Badge.GetComponent<Image>().color = c > 0 ? HzUI.Hex("#FF4D6D") : HzUI.Hex("#1FB5A8");`

`BladeArenaGame.UI.cs:230 / void RefreshBoosters()`	`v.Ring.color = _guide == b ? HzUI.Hex("#FFE066") : new Color(1, 1, 1, 0);`
`BladeArenaGame.UI.cs:244 / void UpdateHud()`	`_hpFill.GetComponent<Image>().color = k > 0.5f ? HzUI.Hex("#4BE08D") : k > 0.25f ? HzUI.Hex("#FFC24B") : HzUI.Hex("#FF5A5A");`
`BladeArenaGame.UI.cs:294 / void UpdateEnemyBars()`	`var fill = HzUI.Panel("F", eb.Bg, HzUI.Hex("#FF6B5B"), 10); eb.FillImg = fill; eb.Fill = fill.rectTransform; Bar(eb.Fill);`
`BladeArenaGame.UI.cs:305 / void UpdateEnemyBars()`	`eb.FillImg.color = v.F.Armor > 0 ? HzUI.Hex("#B9C8DD") : v.F.State == FState.Windup ? HzUI.Hex("#FFD24B") : HzUI.Hex("#FF6B5B");`
`BladeArenaGame.UI.cs:319 / void FloatDamage(Vector3 world, float dmg, bool crit, bool player = false)`	`FloatText(world + Vector3.up * 0.5f, crit ? t + "!" : t, player ? HzUI.Hex("#FF4D4D") : crit ? HzUI.Hex("#FFB21E") : HzUI.Hex("#FFFFFF"), crit ? 76 : 58);`
`BladeArenaGame.UI.cs:363 / void LevelBanner()`	`var edge = HzUI.Panel("Edge", bg.transform, HzUI.Hex("#F2C14E"), 50); edge.sprite = HzUI.RoundRect(50, 6); edge.raycastTarget = false; edge.rectTransform.Stretch(6);`
`BladeArenaGame.UI.cs:365 / void LevelBanner()`	`var t = HzUI.Label("L", bg.transform, "Level " + Level, 92, HzUI.Hex("#FFE7A8")); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(800, 110), new Vector2(0, -18), new Vector2(0.5f, 1));`
`BladeArenaGame.UI.cs:368 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 38, HzUI.Hex("#CFE9FF")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 20), new Vector2(0.5f, 0));`
`BladeArenaGame.UI.cs:374 / void FightBanner() => BigText("FIGHT!", HzUI.Hex("#FFD24B"), Sfx.Fight);`	`void FightBanner() => BigText("FIGHT!", HzUI.Hex("#FFD24B"), Sfx.Fight);`
`BladeArenaGame.UI.cs:376 / void WaveBanner(int wave, int total) => BigText($"WAVE {wave}/{total}", HzUI.Hex("#FF9A6B"), null);`	`void WaveBanner(int wave, int total) => BigText($"WAVE {wave}/{total}", HzUI.Hex("#FF9A6B"), null);`
`BladeArenaGame.UI.cs:446 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", HzUI.Hex("#FFF4DE"), 1.3f, 0.7f);`

`BladeArenaGame.UI.cs:503 / HzPopup ArenaPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var edge = HzUI.Panel("Edge", card.transform, HzUI.Hex("#F2C14E"), 56); edge.sprite = HzUI.RoundRect(56, 8); edge.raycastTarget = false; edge.rectTransform.Stretch(6);`
`BladeArenaGame.UI.cs:561 / void UnlockPopup(int b, Action done)`	`("Awesome!", HzUI.Hex("#1FB5A8"), done));`
`BladeArenaGame.UI.cs:567 / void WinPopup(int lv, int stars, float hp)`	`var p = ArenaPopup("Victory!", $"\n\nLevel {lv} cleared - {Mathf.RoundToInt(hp * 100)}% health left", HzUI.Hex("#1FB5A8"), Art("win_badge"),`
`BladeArenaGame.UI.cs:568 / void WinPopup(int lv, int stars, float hp)`	`("Next Level", HzUI.Hex("#F2A93B"), () => HzPlatform.Interstitial(GameId, () => LoadLevel(Level))));`
`BladeArenaGame.UI.cs:577 / void WinPopup(int lv, int stars, float hp)`	`else { var tx = HzIcons.Icon(row, HzIcons.Star, 110, i < stars ? HzUI.Hex("#FFC93C") : new Color(0.2f, 0.2f, 0.3f, 0.45f)); g = tx; }`
`BladeArenaGame.UI.cs:593 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#4C8DF0"), null),`
`BladeArenaGame.UI.cs:594 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#9B6BFF"), null),`
`BladeArenaGame.UI.cs:595 / void ShowSettingsPopup()`	`("Restart Level", HzUI.Hex("#F2A93B"), () => LoadLevel(Level, true)),`
`BladeArenaGame.UI.cs:596 / void ShowSettingsPopup()`	`("Resume", HzUI.Hex("#1FB5A8"), null));`
`BladeArenaGame.View.cs:15 / class/static declarations`	`new ThemeLook { TopA = "#EFD3A4", TopB = "#E2C08D", Side = "#B9774C", Pillar = "#F4E2C0", Accent = "#D9653B", Sky = "#F3B98C", Abyss = "#C7826A", Sun = "#FFE9CF", AmbSky = "#FFD9B8", AmbEq = "#E5A985", AmbGround = "#8B5A4A", Crumble = "#D6B17F" },`
`BladeArenaGame.View.cs:16 / class/static declarations`	`new ThemeLook { TopA = "#DDF1FA", TopB = "#C6E5F4", Side = "#6E9CC2", Pillar = "#F0FAFF", Accent = "#4FA3D1", Sky = "#A9D8F0", Abyss = "#6FA6CC", Sun = "#F4FBFF", AmbSky = "#D5EEFF", AmbEq = "#A6CBE6", AmbGround = "#4F6F96", Crumble = "#B6D5E6" },`

BladeArenaGame.View.cs:17 / class/static declarations	new ThemeLook { TopA = "#7A6660", TopB = "#6B5852", Side = "#3C2D2D", Pillar = "#94797A", Accent = "#FF7A3D", Sky = "#2B1B24", Abyss = "#B8461F", Sun = "#FFD2B0", AmbSky = "#8A6070", AmbEq = "#B0584A", AmbGround = "#3A1E1E", Crumble = "#8C6F5F", Lava = true },
BladeArenaGame.View.cs:18 / class/static declarations	new ThemeLook { TopA = "#A3D592", TopB = "#8CC47D", Side = "#3F7A55", Pillar = "#E0F2D6", Accent = "#2E9E6E", Sky = "#BFE8CC", Abyss = "#4F8C6C", Sun = "#FFF7DC", AmbSky = "#DDF5E0", AmbEq = "#A8D7B0", AmbGround = "#3F6F52", Crumble = "#A3C58C" },
BladeArenaGame.View.cs:19 / class/static declarations	new ThemeLook { TopA = "#B3A9DE", TopB = "#A196D2", Side = "#584D92", Pillar = "#D6CFF4", Accent = "#8E6BFF", Sky = "#2E2A57", Abyss = "#4A3F8C", Sun = "#E6DDFF", AmbSky = "#9B8FD8", AmbEq = "#7A6AB8", AmbGround = "#2C2558", Crumble = "#9F92C8" },
BladeArenaGame.View.cs:21 / class/static declarations	const string PlayerColor = "#1FB5A8";
BladeArenaGame.View.cs:100 / void SetupLook()	fill.transform.rotation = Quaternion.Euler(30, 150, 0); fill.color = HzUI.Hex("#BFD4FF"); fill.intensity = 0.32f;
BladeArenaGame.View.cs:103 / void SetupLook()	rim.transform.rotation = Quaternion.Euler(20, 180, 0); rim.color = HzUI.Hex("#FFE0C8"); rim.intensity = 0.35f;
BladeArenaGame.View.cs:124 / void MakeMaterials()	_steel = Lit("#C9D3DE", 0.86f, 0.55f);
BladeArenaGame.View.cs:125 / void MakeMaterials()	_gold = Lit("#F2C14E", 0.8f, 0.5f);
BladeArenaGame.View.cs:126 / void MakeMaterials()	_brass = Lit("#D49A3A", 0.75f, 0.45f);
BladeArenaGame.View.cs:127 / void MakeMaterials()	_dark = Lit("#2B2F45", 0.4f);
BladeArenaGame.View.cs:128 / void MakeMaterials()	_skin = Lit("#FFE3C4", 0.35f);
BladeArenaGame.View.cs:129 / void MakeMaterials()	_wood = Lit("#8A5A36", 0.3f);

`BladeArenaGame.View.cs:132 / void MakeMaterials()`	`_tealGlow = Unlit(HzUI.Hex("#5FFFF0") * 2.2f);`
`BladeArenaGame.View.cs:242 / void BuildWorld()`	`var grate = Lit("#4A4E5E", 0.6f, 0.5f);`
`BladeArenaGame.View.cs:318 / void BuildWorld()`	`MeshObj("Body", root, _mBarrel, Lit("#C8433A", 0.45f), true);`
`BladeArenaGame.View.cs:442 / void RebuildPlayerView()`	`var limbM = f.Player ? Lit("#22324F", 0.4f) : Lit(Darken(col, 0.6f), 0.5f, 0.3f);`
`BladeArenaGame.View.cs:466 / void RebuildPlayerView()`	`Part("Cape", v.Body, _mPlate, Lit("#127A72", 0.4f), new Vector3(0, 0.78f, -0.2f), new Vector3(0.78f, 1.05f, 0.22f));`
`BladeArenaGame.View.cs:539 / void RebuildPlayerView()`	`var tc = f.Player ? HzUI.Hex("#B8FFF6") : HzUI.Hex("#FFB070");`
`BladeArenaGame.View.cs:979 / void HitFx(Vector3 at, int victim, bool big)`	`string col = v != null && !v.F.Player ? v.F.P.color : "#FF6B6B";`
`BladeArenaGame.View.cs:988 / void ClashFx(Vector3 at)`	`SpawnSparks(at, SparkMat("#FFE27A", 3f), 16, 1.5f, 0.9f);`
`BladeArenaGame.View.cs:994 / void ArmorOffFx(int victim, int left, Vector3 at)`	`SpawnSparks(at, SparkMat("#DDE6F2", 2.4f), 12, 1.3f);`
`BladeArenaGame.View.cs:1020 / void PlayerHurtFx(Vector3 at)`	`SpawnSparks(at, SparkMat("#FF6B6B", 2.4f), 10, 1.2f);`
`BladeArenaGame.View.cs:1028 / void ExplodeFx(int barrel, Vector3 at)`	`SpawnSparks(at, SparkMat("#FFB040", 3f), 30, 2.2f, 1.4f);`
`BladeArenaGame.View.cs:1043 / void SpawnFx(Vector3 at)`	`SpawnSparks(at + Vector3.up * 0.3f, SparkMat("#FFE27A", 2.2f), 8, 0.8f);`
`BladeArenaGame.View.cs:1050 / void HealFx()`	`SpawnSparks(p, SparkMat("#5BE08A", 2.4f), 14, 0.9f);`
`BladeArenaGame.View.cs:1060 / void VictoryPose()`	`SpawnSparks(basePos + Vector3.up * 1.6f, SparkMat("#F2C14E", 2.6f), 24, 1.6f, 1.2f);`
`BladeData.cs:25 / class/static declarations`	`public string color = "#D9534F";`
`BladeGen.cs:95 / public override string ToString() =>`	`new ProfileData { id = "recruit", name = "Tin Recruit", color = "#E0574F", weapon = 0, hp = 60, dmg = 12, speed = 2.6f, windup = 0.72f, cooldown = 1.4f, aggression = 0.35f, armor = 0, smart = 0.15f, scale = 1f },`

`BladeGen.cs:96 / public override string ToString() =>`	`new ProfileData { id = "duelist", name = "Copper Duelist", color = "#E8913A", weapon = 0, hp = 75, dmg = 14, speed = 3.1f, windup = 0.55f, cooldown = 1.0f, aggression = 0.6f, armor = 0, smart = 0.5f, scale = 1f },`
`BladeGen.cs:97 / public override string ToString() =>`	`new ProfileData { id = "guard", name = "Iron Guard", color = "#7D8DA6", weapon = 0, hp = 80, dmg = 15, speed = 2.6f, windup = 0.62f, cooldown = 1.2f, aggression = 0.5f, armor = 2, smart = 0.4f, scale = 1.05f },`
`BladeGen.cs:98 / public override string ToString() =>`	`new ProfileData { id = "pike", name = "Pike Sentinel", color = "#3FA374", weapon = 1, hp = 70, dmg = 16, speed = 2.5f, windup = 0.6f, cooldown = 1.3f, aggression = 0.5f, armor = 1, smart = 0.5f, scale = 1f },`
`BladeGen.cs:99 / public override string ToString() =>`	`new ProfileData { id = "brute", name = "Brass Brute", color = "#C08A2E", weapon = 2, hp = 160, dmg = 26, speed = 2.1f, windup = 0.95f, cooldown = 1.8f, aggression = 0.5f, armor = 1, smart = 0.3f, scale = 1.3f },`
`BladeGen.cs:100 / public override string ToString() =>`	`new ProfileData { id = "dancer", name = "Shade Dancer", color = "#8456D1", weapon = 3, hp = 60, dmg = 9, speed = 3.8f, windup = 0.32f, cooldown = 0.8f, aggression = 0.8f, armor = 0, smart = 0.7f, scale = 0.92f },`
`BladeGen.cs:101 / public override string ToString() =>`	`new ProfileData { id = "champion", name = "Gilded Champion", color = "#F2C14E", weapon = 0, hp = 300, dmg = 20, speed = 3.0f, windup = 0.5f, cooldown = 0.9f, aggression = 0.7f, armor = 2, smart = 0.8f, scale = 1.45f },`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/BladeArenaGame.UI.cs`, find the displayed literal `"SLASH"`, and replace its displayed value with `"SPIN"`. Keep hierarchy names and technical IDs unchanged. In the portrait/landscape conditional, change both displayed SLASH strings while preserving the [Space] shortcut label. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. BoostName, BoostUnlock, BoostStart, BoostDesc and duration constants are in BladeArenaGame.cs; the HUD consumes them in BladeArenaGame.UI.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`BladeArenaGame.cs:25 / class/static declarations`	`static readonly string[] BoostName = { "Potion", "Whirlwind", "Aegis", "Time Warp" };`
`BladeArenaGame.cs:155 / IEnumerator Intro(string key)`	`ArenaPopup(it.title, it.msg, HzUI.Hex("#D9653B"), Art(it.art), ("Got it!", HzUI.Hex("#1FB5A8"), () => closed = true));`
`BladeArenaGame.cs:454 / IEnumerator LoseFlow(bool fell)`	`ArenaPopup("Defeated!", msg + (_sim.EnemiesLeftTotal() > 0 ? $"\n{_sim.EnemiesLeftTotal()} knight{(_sim.EnemiesLeftTotal() == 1 ? "" : "s")} left standing." : ""), HzUI.Hex("#7A5CC9"), Art("lose_badge"), buttons.ToArray());`
`BladeArenaGame.cs:482 / void OnBooster(int b)`	`if (!Unlocked(b)) { Toast("Unlocks at level " + BoostUnlock[b]); HzAudio.Play(Sfx.Wrong, 0.5f); if (_boost[b] != null) HzTween.Shake(_boost[b].Root, 10, 0.3f); return; }`
`BladeArenaGame.cs:487 / void OnBooster(int b)`	`ArenaPopup("Need a " + BoostName[b] + "?", "Watch a short ad to get one more.", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]),`

`BladeArenaGame.cs:488 / void OnBooster(int b)`	`("Get 1 (ad)", HzUI.Hex("#1FB5A8"), () => HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant().Replace(' ', '_'), ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),`
`BladeArenaGame.cs:552 / public override string DebugCommand(string cmd)`	`case "settings": ShowSettingsPopup(); return "ok";`
`BladeArenaGame.UI.cs:68 / void BuildHud()`	`_levelText = HzUI.Label("Lvl", lp.transform, "Level " + Level, 54, HzUI.Hex("#FFE7A8"));`
`BladeArenaGame.UI.cs:71 / void BuildHud()`	`_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#CFE9FF"), TextAnchor.MiddleCenter, false);`
`BladeArenaGame.UI.cs:94 / void BuildHud()`	`_hpText = HzUI.Label("HpT", hp.transform, "100", 44, Color.white); _hpText.rectTransform.Place(new Vector2(1, 0.5f), new Vector2(110, 80), new Vector2(-62, 0));`
`BladeArenaGame.UI.cs:103 / void BuildHud()`	`_enemyText = HzUI.Label("N", _enemyChip, "", 46, Color.white); _enemyText.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(120, 70), new Vector2(150, 12));`
`BladeArenaGame.UI.cs:104 / void BuildHud()`	`_waveText = HzUI.Label("W", _enemyChip, "", 26, HzUI.Hex("#CFE9FF"), TextAnchor.MiddleCenter, false); _waveText.fontStyle = FontStyle.Bold;`
`BladeArenaGame.UI.cs:119 / void BuildHud()`	`_slashBtn = HzUI.Button("Slash", bar, null, HzUI.Hex("#F2C14E"), Color.white, null, 40, (int)(ss / 2));`
`BladeArenaGame.UI.cs:129 / void BuildHud()`	`var cap = HzUI.Label("Cap", srt, portrait ? "SLASH" : "SLASH [Space]", portrait ? 38 : 30, Color.white);`
`BladeArenaGame.UI.cs:152 / void BuildHud()`	`_tipText = HzUI.Label("T", _tipPanel, "", portrait ? 38 : 40, Ink); _tipText.rectTransform.Stretch(16);`
`BladeArenaGame.UI.cs:183 / static void SetBar(RectTransform r, float k)`	`v.Btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(id), 40, (int)(size / 2));`

`BladeArenaGame.UI.cs:208 / static void SetBar(RectTransform r, float k)`	`v.BadgeText = HzUI.Label("N", bd.transform, "", 34, Color.white); v.BadgeText.rectTransform.Stretch();`
`BladeArenaGame.UI.cs:210 / static void SetBar(RectTransform r, float k)`	`v.Caption = HzUI.Label("Cap", v.Root, "", 28, Color.white);`
`BladeArenaGame.UI.cs:227 / void RefreshBoosters()`	`v.BadgeText.text = c > 0 ? c.ToString() : "+";`
`BladeArenaGame.UI.cs:229 / void RefreshBoosters()`	`v.Caption.text = un ? (IsPortrait ? BoostName[b] : $"{BoostName[b]} [{b + 1}]") : "Level " + BoostUnlock[b];`
`BladeArenaGame.UI.cs:274 / void UpdateEnemyCount(bool punch)`	`_waveText.text = _sim.Waves > 1 ? $"Wave {_sim.WaveIdx + 1}/{_sim.Waves}" : "Knights";`
`BladeArenaGame.UI.cs:325 / void FloatText(Vector3 world, string text, Color c, int size = 56)`	`var l = HzUI.Label("Float", _labels, text, size, c);`
`BladeArenaGame.UI.cs:365 / void LevelBanner()`	`var t = HzUI.Label("L", bg.transform, "Level " + Level, 92, HzUI.Hex("#FFE7A8")); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(800, 110), new Vector2(0, -18), new Vector2(0.5f, 1));`
`BladeArenaGame.UI.cs:368 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 38, HzUI.Hex("#CFE9FF")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 20), new Vector2(0.5f, 0));`
`BladeArenaGame.UI.cs:382 / void BigText(string text, Color c, string sfx)`	`var l = HzUI.Label("Big", _hud, text, 150, c);`
`BladeArenaGame.UI.cs:434 / void RefreshGuide()`	`ShowTip($"New helper: {BoostName[_guide]}! Tap it{key} to try it, this one is free.", 0);`
`BladeArenaGame.UI.cs:446 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", HzUI.Hex("#FFF4DE"), 1.3f, 0.7f);`
`BladeArenaGame.UI.cs:528 / HzPopup ArenaPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 72, Color.white); t.rectTransform.Stretch(6);`
`BladeArenaGame.UI.cs:533 / HzPopup ArenaPopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 42, Ink, TextAnchor.MiddleCenter, false);`

`BladeArenaGame.UI.cs:560 / void UnlockPopup(int b, Action done)`	`ArenaPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!" + key, HzUI.Hex(BoostColor[b]), Art(BoostArt[b]),`
`BladeArenaGame.UI.cs:567 / void WinPopup(int lv, int stars, float hp)`	`var p = ArenaPopup("Victory!", $"\n\nLevel {lv} cleared - {Mathf.RoundToInt(hp * 100)}% health left", HzUI.Hex("#1FB5A8"), Art("win_badge"),`
`BladeArenaGame.UI.cs:592 / void ShowSettingsPopup()`	`var p = ArenaPopup("Settings", "", Navy, null,`
`BladeArenaGame.UI.cs:593 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#4C8DF0"), null),`
`BladeArenaGame.UI.cs:594 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#9B6BFF"), null),`
`BladeArenaGame.UI.cs:606 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`BladeArenaGame.UI.cs:607 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/bladearena through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Game effects are under Resources/bladearena/Sfx. Replace each clip without changing its load key or importer settings. The shared settings dialog has separate sound and music toggles; this package has no dedicated music loop under Resources/bladearena/Music. Kit UI uses supplied fonts; preserve licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.bladearena`, version, scene orientation and signing. The build command writes `Builds/Android/bladearena.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/bladearena/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.